# AKS Perception Progress Report

Oct 21, 2026


### Summary

The OAK-D runs a custom 3-class detector (cone, person, kart) at 17 FPS at 512x288. Fine-tuning on 220 self-collected RC car frames (merged into the kart class) raised RC car mAP50 from 0 to 0.72 and full-val mAP50 from 0.67 to 0.77. Lane detection for the tape track is planned but not started. Next: build lane detection, then bring up Donkey Car on the Jetson for end-to-end driving on the 1/10 car.


### Accomplished

### Requirements and research

- Reviewed the AKS rulebook: all compute stays on the kart, a software urgent stop and a mechanical brake are required, and practice-lap data can be used in the race.
- At 30 mph (13.4 m/s), the kart needs to detect obstacles about 25-32 m ahead. This assumes 3-4 m/s² braking and about 190 ms latency, both still to be measured.
- The wide OAK-D sees a person at about 5-7 m (estimate), so it is the near-field classifier. Long range has to come from lidar.

### Cone detector

Trained YOLO11n at 416 in Colab on FSOCO (converted to YOLO format), then exported it to the OAK-D as an RVC2 archive through Luxonis HubAI. Pre-built YOLOv6n struggled with cones because COCO has no cone class.

### Merged 3-class detector

One model with classes 0 cone, 1 person, 2 kart, trained in Colab starting from COCO-pretrained yolo11n.pt.

- Each source dataset labels only its own class, so the other models pseudo-labeled the missing classes. This keeps unlabeled people and cones from being learned as background.
- Exported through HubAI. On the OAK-D it ran at about 7 FPS at 640x640 and 17 FPS at 512x288.

### RC car data and fine-tune

1. Recorded 3 sessions of the team's RC car at camera mount height and kept every 5th frame: 220 frames.

2. Pre-labeled with YOLO-World (zero-shot) and reviewed every box in Label Studio: 267 boxes.
3. Merged into the kart class (ID 2), split by recording session: 154 train, 66 val (15:20 session held out), 12 negatives.
4. Fine-tuned locally on the GPU from the current 3-class weights, on the full merged set so cones and people are not forgotten (about 40 epochs, early stopping after 15).

The fine-tune taught the model RC cars from scratch and improved kart detection overall:

| Val set | Model | mAP50 | mAP50-95 | Kart mAP50 | Kart mAP50-95 |
|---|---|---|---|---|---|
| Full val | Old | 0.672 | 0.463 | 0.617 | 0.445 |
| Full val | New | **0.767** | **0.529** | **0.859** | **0.621** |
| RC-only val | Old | 0.000 | 0.000 | 0.000 | 0.000 |
| RC-only val | New | **0.723** | **0.526** | **0.723** | **0.526** |

The old model never detected the RC car. RC-only val is the held-out 15:20 session, so the 0.723 mAP50 is on footage the model never trained on.

**Datasets used**

| Dataset | Used for | Size and license |
|---|---|---|
| FSOCO | Cones | Converted to YOLO format |
| AKS traffic cone dataset (Roboflow) | Cones | Merged with FSOCO |
| COCO | Persons | About 4k images, pulled with fiftyone |
| Roboflow go_kart | Karts | 499 images, CC BY 4.0 (credit required) |
| Own OAK-D footage | RC cars (as kart) | 220 frames from 3 sessions |

An external Roboflow RC car set was dropped in favor of the team's own footage, which matches the real camera height and angle.

## Status and open issues

The fine-tuned model is trained, evaluated, and running on the OAK-D locally.

- **Frame rate:** 512x288 raised the detector from 7 to 17 FPS, still below the 25-30 FPS target. Recall on small, far-away RC cars at that size still needs checking.

- **Box alignment:** the detector input (16:9) and a future lane frame (4:3) may differ in crop. YOLO boxes must be checked against the lane frame before they are used to mask lanes.
- **Dataset size:** 267 RC car boxes from 3 sessions is a first pass. More distance and lighting variety may be needed.

## Next steps

1. **Check cone and person accuracy:** overall mAP rose, but confirm per-class cone and person mAP did not drop behind the kart gain.
2. **Push frame rate toward 25 FPS:** the fine-tuned model runs on the OAK-D at 512x288 (17 FPS). Try a smaller 16:9 input such as 416x234. Note that 416x416 has more pixels than 512x288, so it would likely be slower. Any smaller size must be re-checked for accuracy and detection range, especially on far-away cones and RC cars, which shrink to a few pixels first.
3. **Start lane detection:** build the classical tape-line detector (HSV threshold, bird's-eye warp, sliding-window fit), calibrate it on the final mount, tune the tape color, then wire lane output into the OAK pipeline with YOLO boxes masked out. Tape lines only cover the 1/10 track; for sidewalks and the full-size track's pavement edge, train a drivable-area segmentation model (YOLO11n-seg, labeled as polygons in Label Studio), with lidar curb detection as a later cross-check.
4. **Set up Donkey Car on the Jetson Orin Nano.** Its docs target Orin with JetPack 5.0.2, Python 3.8 and TensorFlow 2.9 (Donkey Car Jetson guide). Confirm our JetPack version first.
5. **Follow the Donkey Car workflow:** create the car app, calibrate steering and throttle, drive manually to collect data, then train an autopilot (user guide).
6. **Connect perception to Donkey Car** as custom parts. Detections feed obstacle slow-down or stop; the lane target point feeds steering as a check on, or fallback for, the learned autopilot.

Open question: whether Donkey Car's learned driving replaces the lane-following controller, or runs alongside it as a comparison.

Stats of current object detection model:

| Class | Images | Instances | Precision | Recall | mAP50 | mAP50-95 |
|---|---|---|---|---|---|---|
| All | 3,095 | 18,319 | 0.887 | 0.690 | 0.767 | 0.529 |
| Cone | 2,787 | 15,646 | 0.891 | 0.650 | 0.730 | 0.467 |
| Person | 798 | 2,470 | 0.803 | 0.617 | 0.714 | 0.498 |
| Kart | 183 | 203 | 0.966 | 0.803 | 0.859 | 0.621 |